

Efficient and Fair Allocation on Graphs: From Orientation to Position-Aware Valuations

Bo Li*

Department of Computing
The Hong Kong Polytechnic
University
Hong Kong, China
comp-bo.li@polyu.edu.hk

Ankang Sun*[†]

School of Computer Science and
Technology
Shandong University
Qingdao, China
ankang.sun@sdu.edu.cn

Shiji Xing*

Department of Computing
The Hong Kong Polytechnic
University
Hong Kong, China
shi-ji.xing@connect.polyu.hk

Abstract

In traditional resource/task allocation models, an agent's value on an item is fixed regardless of when, where, or how the agent consumes the item. In this work, we introduce a more general framework involving a set of positions (for example, roles within a company), where agents have valuations that depend on the specific positions they are assigned. This setting is well motivated as the agent faces different resources and support at different positions, and thus may need varying levels of effort to complete the item, supposing the items are chores. We further consider the constrained setting when each item can only be allocated to a certain set of positions (e.g., a research project cannot be assigned to an administrative staff). We particularly consider the case when the constraints form a graph where edges are items and vertices are positions, so an item can only be allocated to an incident position, which is known as the orientation problem [Christodoulou et al. EC 2023]. In this paper, fairness is measured by maximin share (MMS), and efficiency is measured by Pareto and social optimality. We present a complete set of results on the computational complexity and approximation algorithms for computing efficient and fair allocations.

CCS Concepts

• Theory of computation → Approximation algorithms analysis; Complexity classes.

Keywords

Fair Division, Graphs, Maximin Share, Orientation

ACM Reference Format:

Bo Li, Ankang Sun, and Shiji Xing. 2026. Efficient and Fair Allocation on Graphs: From Orientation to Position-Aware Valuations. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792590>

*The author names are listed in alphabetical order, and all authors are corresponding authors. This work is supported in part by the Hong Kong SAR Research Grants Council (Grant No. PolyU 15224823) and the Guangdong Basic and Applied Basic Research Foundation (Grant No. 2024A1515011524).

[†]Part of the work was done when Ankang Sun was a postdoctoral fellow at The Hong Kong Polytechnic University.



This work is licensed under a Creative Commons Attribution 4.0 International License. [WWW '26, Dubai, United Arab Emirates](https://creativecommons.org/licenses/by/4.0/)

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792590>

1 Introduction

1.1 Position-Aware Valuations

Consider an urban inspection task scenario, where multiple monitoring stations are responsible for inspecting urban infrastructure, such as roads, bridges, and power grids. Inspection tasks, such as patrol routes, need to be allocated to nearby stations. Each inspector needs to be assigned to a monitoring station and is responsible for handling the inspection tasks in its vicinity. As different stations have varying resources (such as access to transportation and drone support), the cost of performing the same task can differ depending on the station to which an inspector is assigned. Different inspectors have varying expertise in specific inspection tasks, resulting in different execution costs for the same task (when placed at the same station). Our objective is to compute an assignment that is fair and efficient. This problem falls under the umbrella of fair (and efficient) allocation of resources, which has been very widely studied, especially in the past decade when computation issues attracted significant attention [4, 36].

However, the above problem cannot be exactly captured by existing models where an agent's cost on a task is fixed and does not depend on when, where, and how the agent will complete the task; see, e.g., [14, 31]. In our problem, an agent may face different costs to complete the same task if she is in different positions, due to different available resources and facilities. For example, it is easier for a treasurer to deal with financial tasks than the other roles in the committee. That is, in our problem, the cost functions are *position-aware*. In this work, we formally introduce the model of fair (and efficient) allocation with position-aware valuations. Our model is more general and when all positions are the same, it degenerates to the traditional one. Besides position-aware valuations, our model also provides interfaces to incorporate other real-world factors, abstracted as constraints. For example, in many real-world applications, a task can only be feasibly assigned to a certain subset of roles, due to the nature of the task and the roles. Similarly, an agent may only be capable of taking a certain subset of roles, due to different abilities and expertise.

1.2 The Graph Orientation Model

In a graph orientation problem, we are given a graph where items are edges and each vertex corresponds to an agent. It is required that every edge/item is only assigned to one of the two incident vertices/agents. Such a model captures many real-world applications. Consider the problem of assigning home courts in sports leagues like NBA and NFL. Games between two teams must be hosted at

the home court of one of the two teams. Similar problems also arise in, for example, the scheduling of shuttle buses between two terminals (which terminal should each bus park in the evening). Christodoulou et al. [15] initialized the study of graph orientation in the context of fair division and follow-up works [17, 41] have considered envy-free related notions in this model. Our position-aware model generalizes their model in the way that the agents are not fixed at vertices. Instead, the vertices represent positions so that later we can assign agents to the vertices.

In addition to their practical applications, position-aware valuations offer a natural theoretical framework for studying the graph orientation of chores, where the direct dual formulation of goods lacks natural interpretations [41]. Assume agents are fixed on vertices and have zero cost for edges not incident to them. Then allocating non-incident edges to agents would seem optimal, since they incur no cost – but doing so violates the orientation constraints. Alternatively, one could interpret non-incident items as jobs the agent is not capable of performing. However, this raises new issues: if an agent is not capable of performing a job, why should they envy someone who is? The position-aware model offers a natural resolution, where jobs are assigned to positions, and agents can only perform a job if they are assigned to the corresponding position.

1.3 Technical Results

Our technical results are obtained for the following model, where we have n agents, n posts (or positions), and m items (or tasks). The items and posts form a graph, with posts being vertices and items being edges. It is required that each item/edge can only be feasibly assigned to one of the two incident posts/vertices. We are concerned with both efficiency and fairness of assignments. For efficiency, we study social cost minimization and Pareto optimality (PO). Social cost minimization aims for the assignment where the sum of all agents' costs is minimized. Such an assignment is also called socially optimal or optimal for short. An assignment is PO if we cannot decrease an agent's cost without hurting any other agent. It is easy to see that an optimal assignment must be PO. For fairness, we particularly focus on the notion of maximin share (MMS) [13]. Informally, the maximin share of an agent is the minimum cost that the agent can obtain by partitioning the items into n parts and taking the part with the maximum cost, where n is the number of agents. The approximations of MMS has been extensively studied, e.g., [2, 7, 20] for goods (items with positive valuations) and [23, 24] for chores (items with negative valuations).

We say the valuations are *symmetric* if the costs of items are the same to the two incident positions, and *binary* if all costs are either 0 or 1. Besides general graphs, we also consider some special graphs, such as *stars*, *paths*, and *cycles*. These graphs are well-motivated in our model. A star graph captures the scenario where a group leader coordinates all tasks with other members. A path graph means a set of tasks that need to be processed sequentially, and a cycle graph means these tasks are processed sequentially and repeatedly. Our results are summarized in Table 1.

We first consider the problem of computing efficient assignments. We prove that:

- Firstly, computing a PO or finite approximation of social cost minimization assignment is NP-hard even for symmetric cost

functions with binary values, and the graph G is (i) a union of vertex-disjoint paths of length at most 3, (ii) a path, and (iii) a cycle.

- Secondly, for general cost functions and a graph with a constant number of stars, an assignment minimizing social cost (which implies PO) can be found in polynomial time.
- Thirdly, for general cost functions and a single star graph, an assignment minimizing the egalitarian social cost can be found in polynomial time.

We also consider MMS fair assignments and obtain the following results.

- Firstly, computing the MMS value in general is NP-hard, while for binary cost functions, the problem is in P.
- Secondly, for binary valuations, there is no finite approximation of MMS assignment, while for symmetric cost functions, a 2-approximate MMS assignment can be found efficiently.
- Thirdly, for a path, star, or cycle, with general cost functions, the MMS value can be computed in polynomial time. Moreover, an exact MMS assignment always exists and can be computed efficiently.

1.4 Other Related Work

Fair orientation in graphs. The constraints we consider between items and posts are motivated by the line of research on envy-free orientations in graphs, initiated by Christodoulou et al. [15]. Christodoulou et al. considered the case of goods, where agents are fixed at vertices and only interested in edges incident to it, and proved that deciding the existence of an EFX orientation is NP-hard. Zeng and Mehta [40] later characterized the structure of graphs so that an EFX orientation is guaranteed to exist. Afshinmehr et al. [1] and Hsu [22], respectively, extended the research to multigraphs. Amanatidis et al. [5] and Kaviani et al. [29] designed algorithms to compute approximate EFX allocations. Regarding the weaker notion EF1, Deligkas et al. [17] proved that an EF1 orientation always exists¹. Christodoulou and Mastrakoulis [16] proved the existence of MMS allocations and extended these results to broader valuation classes, including XOS and subadditive valuations. Finally, Zhou et al. [41] extended the model of goods to the models of chores and mixtures of goods and chores when the value of each item can be positive or negative.

Fair allocation with constraints. Many other types of constraints have been investigated as well, such as hereditary [25, 34], budget-feasible [6, 38], cardinality [11, 27], conflicting [26], and scheduling [32]. Some of these constraints can be incorporated into valuations, leading to the study of subadditive, fractionally subadditive, and submodular valuations [3, 21]. A survey of constrained fair allocation can be found in [37]. The above works are focused on goods. Although constrained chore allocation is studied less comprehensively, there are several recent works extending budget-feasible [18], scheduling constraints [30], and general valuations [33] to chores. There is another line of research that defines constraints using graphs, where the items are vertices in a graph and each

¹Deligkas et al. [17] proved the existence of an EF1 orientation in a broader setting, not limited to graph orientation, when each item can be feasibly allocated to an arbitrary set of agents.

Setting	General	Special Valuations		Special Graphs		
		Symmetric	Binary	Star	Path	Cycle
PO	NP-h (T.4)	NP-h (T.4)	NP-h (T.4)	P (T.7)	NP-h (C.5)	NP-h (C.5)
UtilSW	NP-h (C.6)	NP-h (C.6)	NP-h (C.6)	P (T.7)	NP-h (C.6)	NP-h (C.6)
EgalSW	NP-h (C.6)	NP-h (C.6)	NP-h (C.6)	P (T.8)	NP-h (C.6)	NP-h (C.6)
MMS	NP-h (T.9)	NP-h (T.9)	P (T.10)	P (T.15)	P (T.16)	P (T.11)
α -MMS	∞ (T.12)	2 (T.14)	∞ (T.12)	1 (T.15)	1 (T.16)	∞ (T.12)

Table 1: Main Results. PO, UtilSW, and EgalSW are the problems of computing a PO assignment or an assignment that minimizes the utilitarian or egalitarian social cost. MMS means computing the MMS value for an agent and α -MMS means the existence of an α -approximate MMS assignment. P and NP-h mean polynomial-time solvable and NP-hard.

agent is required to receive a connected subgraph [10, 12, 28]. Note that in this model, the agents are not elements of the graph.

2 Preliminaries

For any integer $k \geq 1$, denote by $[k] = \{1, \dots, k\}$. We consider a set M of jobs and a set $P = \{P_1, \dots, P_n\}$ of n positions where each P_i is relevant to a set of jobs $R_i \subseteq M$ and $\bigcup_{j=1}^n R_j = M$. The jobs in R_i are also referred to as *relevant* to position P_i . The intersection of the sets of relevant jobs of different positions can be non-empty. There is a set of n agents $N = \{1, \dots, n\}$. Our task is to distribute agents to positions such that each position is filled by one agent, and also allocate relevant jobs to positions in a fair and efficient manner. By assigning agents to positions, we find a one-to-one mapping from agents to the positions, denoted by $\sigma : [n] \rightarrow [n]$, such that agent i is assigned to position $P_{\sigma(i)}$. Further, we allocate each position P_i a set of jobs A_i that are relevant to it, i.e., $A_i \subseteq R_i$. Throughout the paper, A_i refers to the set of jobs allocated to P_i , rather than to agent i . It is required that all jobs are completed, and each job is allocated to one position, i.e., for any $i \neq j$, $A_i \cap A_j = \emptyset$ and $\bigcup_{j \in [n]} A_j = E$. Let $\mathcal{A} = (A_1, \dots, A_n)$ represent the allocation of jobs to positions, and an *assignment* is denoted by the tuple $(\mathcal{A}, \sigma)$. Note that for all the assignments we consider in this paper, it is required that $A_i \subseteq R_i$ for all $P_i \in P$.

In this paper, we particularly focus on the case where each job is relevant to two positions so that a job-position instance can be represented by a graph $G = (V, E)$ where each post P_i is a vertex $v_i \in V$ and each job $e_j = (v, v') \in E$ is an edge between v and v' in the graph. We do not consider multi-graphs. Thus, R_i also refers to the set of edges incident to vertex v_i . Throughout the paper, we use P_i and v_i , positions and vertices, jobs and edges, interchangeably, when there is no confusion.

We consider a general cost function where an agent may have different cost on the same job when she is placed at different positions. Formally, for any agent i at position P_j , the cost function of i is $c_i^j : 2^{R_j} \rightarrow \mathbb{N}^+ \cup \{0\}$. For the computational purpose, we assume that cost functions take non-negative integral values. For simplicity, denote $c_i^j(\{e\})$ by $c_i^j(e)$. The cost functions are additive so that for any i, j and $S \subseteq R_j$, $c_i^j(S) = \sum_{e \in S} c_i^j(e)$. A cost function c_i^j is *binary* if for each $e \in E$, $c_i^j(e) \in \{0, 1\}$. We also consider *symmetric* cost functions where the cost of jobs does not differ from different positions, i.e., for any P_j, P_l and $e \in R_j \cap R_l$, $c_i^j(e) = c_i^l(e)$. In the setting of symmetric cost functions, we use c_i to refer to the cost function of agent i .

2.1 Solution Concepts

Below we define the efficiency and fairness criteria considered in this paper. We present the definitions for general cost functions, which also apply to symmetric cost functions.

DEFINITION 1 (PARETO OPTIMAL). An assignment $(\mathcal{A}, \sigma)$ Pareto dominates another assignment $(\mathcal{A}', \sigma')$ if for any $i \in [n]$, $c_i^{\sigma(i)}(A_{\sigma(i)}) \leq c_i^{\sigma'(i)}(A'_{\sigma'(i)})$ and at least one inequality is strict. The assignment $(\mathcal{A}, \sigma)$ is Pareto optimal (PO) if no assignment Pareto dominates it.

DEFINITION 2 (SOCIAL COST FUNCTIONS). Given an assignment $(\mathcal{A}, \sigma)$, its utilitarian social cost is $\sum_{i \in [n]} c_i^{\sigma(i)}(A_{\sigma(i)})$ and its egalitarian social cost is $\max_{i \in [n]} c_i^{\sigma(i)}(A_{\sigma(i)})$.

We now introduce the notion of *maximin share* (MMS). Denote by $\Pi_n(E)$ the set of n -partitions of E and moreover for any $(X_1, \dots, X_n) \in \Pi_n(E)$, each X_i is a subset of R_i . The maximin share of agent i is

$$\text{MMS}_i = \min_{(X_1, \dots, X_n) \in \Pi_n(E)} \max_{j \in [n]} c_i^j(X_j).$$

DEFINITION 3 (α -MMS). For any $\alpha \geq 1$, an assignment $(\mathcal{A}, \sigma)$ is α -MMS if for any $i \in [n]$, $c_i^{\sigma(i)}(A_{\sigma(i)}) \leq \alpha \cdot \text{MMS}_i$.

3 Computing Efficient Assignments

In this section, we study the problems of computing PO assignment and computing the social cost minimization assignment. We establish NP-hardness by reductions from 2P2N-3SAT, known to be NP-complete [8, 39]. A 2P2N-3SAT instance contains a Boolean formula in conjunctive normal form consisting of n variables $\{x_i\}_{i \in [n]}$ and m clauses $\{C_j\}_{j \in [m]}$ with three distinct literals each, and moreover, each variable appears exactly twice as a positive literal and exactly twice as a negative literal.

We first show hardness result for computing PO assignment, even when cost functions are symmetric and binary additive. Since a PO assignment always exists, we cannot employ the standard approach of showing that a decision problem is NP-hard via many-one reductions. Instead, we show, by a Turing reduction, that a polynomial-time algorithm computing a PO assignment can be used to solve an NP-complete problem in polynomial time.

THEOREM 4. For symmetric binary cost functions, it is NP-hard to compute a PO assignment when G is a union of vertex-disjoint paths, each of length at most 3.

PROOF. We give a Turing reduction from the 2P2N-3SAT. Given an instance of 2P2N-3SAT with n variables and m clauses, we create a job-position instance where G has $4n + 5m$ vertices and agents, and $5n + m$ edges/jobs. Moreover, G is a union of vertex-disjoint paths of length at most 3 each.

With a slight abuse of notations, for each variable x_i , create two agents $x_i, \bar{x}_i$ and two vertices $P_{x_i}, P_{\bar{x}_i}$ corresponding to $x_i, \bar{x}_i$ respectively. For each clause C_j , let C_j^1, C_j^2, C_j^3 be respectively the first, second and third literals of C_j . Then for C_j , we create three agents C_j^1, C_j^2, C_j^3 and three vertices $P_{C_j^1}, P_{C_j^2}, P_{C_j^3}$ corresponding to these agents. There are also $2n + 2m$ dummy agents. The detailed construction of G is presented as follows:

- For each variable x_i , create four agents $x_i, \bar{x}_i, D_{x_i}^1, D_{x_i}^2$ where $D_{x_i}^1, D_{x_i}^2$ are two dummy agents. Create vertices $P_{x_i}, P_{\bar{x}_i}$ (corresponding to agents $x_i, \bar{x}_i$) and vertices $P_{x_i}^{d_1}, P_{x_i}^{d_2}$ (corresponding to dummy agents $D_{x_i}^1, D_{x_i}^2$). Create edge $(P_{x_i}^{d_1}, P_{x_i}^{d_2})$.
- For each C_j , create five agents $C_j^1, C_j^2, C_j^3, D_{C_j}^1, D_{C_j}^2$ where $D_{C_j}^1, D_{C_j}^2$ are dummy agents. Create vertices $P_{C_j^1}, P_{C_j^2}, P_{C_j^3}$ (corresponding to agents C_j^1, C_j^2, C_j^3) and vertices $P_{C_j}^{d_1}, P_{C_j}^{d_2}$ (corresponding to dummy agents $D_{C_j}^1, D_{C_j}^2$). Create edge $(P_{C_j}^{d_1}, P_{C_j}^{d_2})$. In addition, for every literal vertex, create an edge between it and its corresponding variable vertex. For example, if the first literal of clause C_j is x_i (resp., $\bar{x}_i$), then create edge $(P_{C_j^1}, P_{x_i})$ (resp., $(P_{C_j^1}, P_{\bar{x}_i})$).

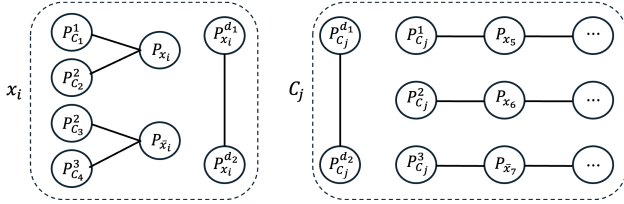


Figure 1: The illustration of the created graph in the proof of Theorem 4. Consider variable x_i and clause C_j . Suppose x_i is the first literal (based on a deterministic order) in C_1 and the second literal in C_2 , and $\bar{x}_i$ is the second literal in C_3 and the third literal in C_4 . The clause C_j is $C_j = (x_5 \vee x_6 \vee \bar{x}_7)$.

We have completed the construction of G , illustrated in Figure 1. Every pair of dummy vertices form a path of length 2. Each P_{x_i} (resp., $P_{\bar{x}_i}$) has degree 2 as variable x_i (resp., $\bar{x}_i$) appears exactly twice in the 2P2N-3SAT instance. Moreover, each vertex $P_{C_j^t}$ has degree 1. Thus, G is a union of vertex-disjoint paths, each of length 2 or 3. Next, we present the cost functions.

- For any $i \in [n]$, agent x_i (resp., $\bar{x}_i$) has cost 0 for each edge incident to P_{x_i} (resp., $P_{\bar{x}_i}$). In addition, both agents x_i and $\bar{x}_i$ have cost 0 for edge $(P_{x_i}^{d_1}, P_{x_i}^{d_2})$.
- For any $j \in [m]$ and any $t \in [3]$, agent C_j^t has cost 0 for each edge incident to $P_{C_j^t}$ and for edge $(P_{C_j}^{d_1}, P_{C_j}^{d_2})$.
- For an agent and an edge, if the cost of the edge for the agent is not covered by the above two sentences, then the cost is 1.

For the sake of contradiction, suppose that there exists an algorithm ALG that computes a PO assignment in polynomial time. We show how to use ALG to solve 2P2N-3SAT. Given a 2P2N-3SAT instance, we create a job-position instance and run ALG , resulting in a PO assignment $(\mathcal{A}, \sigma)$. In the following, we prove that 2P2N-3SAT has a solution if and only if each agent has cost zero in $(\mathcal{A}, \sigma)$.

Suppose that in the returned assignment $(\mathcal{A}, \sigma)$, every agent receives cost 0. We will construct a solution to the 2P2N-3SAT instance. For a pair of agents x_i and $\bar{x}_i$, one of them must receive edge $(P_{x_i}^{d_1}, P_{x_i}^{d_2})$ in $(\mathcal{A}, \sigma)$; otherwise, the agent receiving that edge has cost 1. Accordingly, at least one of x_i and $\bar{x}_i$ must be placed at either $P_{x_i}^{d_1}$ or $P_{x_i}^{d_2}$, and equivalently, at most one of x_i and $\bar{x}_i$ can be placed at their corresponding vertices $P_{x_i}, P_{\bar{x}_i}$. If in $(\mathcal{A}, \sigma)$, exactly one of them is placed at the corresponding vertex (say $\bar{x}_i$ is at $P_{\bar{x}_i}$), we set $\bar{x}_i$ to True. If neither of them is placed at the corresponding vertices, we set x_i to True. Then for each x_i and its negation $\bar{x}_i$, exactly one of them is set to True, and the assignment is valid. Next we show that each clause has at least one positive literal. In $(\mathcal{A}, \sigma)$, edge $(P_{C_j}^{d_1}, P_{C_j}^{d_2})$ must be assigned to one of agents C_j^1, C_j^2, C_j^3 , and without loss of generality, suppose that agent C_j^1 receives that edge. Then, C_j^1 is not placed at vertex $P_{C_j^1}$. Let P_x be the (unique) vertex incident to $P_{C_j^1}$; recall that the degree of $P_{C_j^1}$ is 1. Then the agent x who corresponds to P_x must be placed at vertex P_x and receives edge $(P_{C_j^1}, P_x)$. According to the above-mentioned rule, variable x is set to True. Therefore, each clause C_j has a positive literal and the 2P2N-3SAT instance has a solution.

For the other direction, suppose that there is a solution π to the instance of 2P2N-3SAT, but towards a contradiction, assume that in assignment $(\mathcal{A}, \sigma)$, some agent receives positive cost. Next, we create another assignment $(\mathcal{A}', \sigma')$ based on π and show that $(\mathcal{A}', \sigma')$ Pareto dominates $(\mathcal{A}, \sigma)$, which derives the desired contradiction. For every pair of $x_i, \bar{x}_i$, if variable x_i is set to True, place agent x_i at P_{x_i} and agent $\bar{x}_i$ at $P_{x_i}^{d_1}$; otherwise, place agent x_i at $P_{x_i}^{d_1}$ and agent $\bar{x}_i$ at $P_{\bar{x}_i}$. Then assign agents x_i and $\bar{x}_i$ all edges incident to them. For each C_j , since π is a solution, there exists at least one of vertices $P_{C_j^1}, P_{C_j^2}, P_{C_j^3}$ whose incident edge has already been allocated; suppose that $P_{C_j^1}$ is such a vertex. Then place agent C_j^1 at vertex $P_{C_j^1}$ and place agents C_j^2, C_j^3 at vertices $P_{C_j^2}, P_{C_j^3}$ respectively. Assign agents C_j^t 's all edges incident to them. At this point, all edges are allocated. Lastly, we arbitrarily distribute dummy agents to unfilled vertices. In the resulting assignment $(\mathcal{A}', \sigma')$, each agent has cost 0. Hence, $(\mathcal{A}', \sigma')$ Pareto dominates $(\mathcal{A}, \sigma)$, which contradicts the definition of ALG .

For a given assignment, whether each agent has cost can be checked in polynomial time, then the above-established equivalence implies that ALG can be used to solve 2P2N-3SAT. Therefore, the problem of computing a PO assignment is NP-hard. $\square$

Starting from the job-position instance created in the above proof, add new edges (only) in an arbitrary way such that the resulting graph becomes a path or a cycle. Let agents have cost 0 for every new edge. Such a reduction directly results in the following corollary.

COROLLARY 5. *For symmetric binary cost functions, it is NP-hard to compute a PO assignment when G is a path or a cycle.*

The above two reductions indicate that one cannot, in polynomial time, decide whether there exists an assignment such that each agent has cost 0, and hence, rule out the possibility of any finite-factor multiplicative approximation.

COROLLARY 6. *For symmetric binary cost functions, it is NP-hard to compute an assignment with finite multiplicative approximation for the utilitarian or egalitarian cost minimization assignment, even when G is (i) a union of vertex-disjoint paths of length at most 3 each, (ii) a path, and (iii) a cycle.*

We next study star graphs where the results are more positive. Let $K_{1,n-1}$ be the star graph with one internal vertex and $n - 1$ leaves.

THEOREM 7. *When G is a star and agents have general cost functions, we can compute the utilitarian social cost minimization assignment in polynomial time.*

PROOF. For a star graph, i.e., $G = K_{1,n-1}$, without loss of generality, let P_n be the internal vertex and $\{P_j\}_{j \in [n-1]}$ be the set of leaves. Let W be an integer with $W \gg \max_{t,q,e} |c_t^q(e)|$. We compute n candidate assignments and prove that the optimal one among these candidates minimizes utilitarian social cost among all assignments.

For each agent $i \in [n]$, we construct a candidate assignment as follows. Place agent i at the internal vertex P_n and then compute a complete assignment through maximum weight matching. In particular, create a weighted bipartite $G' = (X \cup Y, E')$ where X consists of $n - 1$ vertices, each named as x_i , corresponding to agent i , and one vertex x_k for each agent $k \in [n] \setminus \{i\}$, i.e., $|X| = 2n - 2$. For Y , there is a vertex y_j for every job e_j , i.e., $|Y| = n - 1$. For any $x_k \in X$ and $y_j \in Y$, there is an edge (x_k, y_j) in G' . Moreover the weight of edge (x_k, y_j) is $W - c_{kj}$ where if $k \neq i$, c_{kj} is equal to the cost of job e_j for agent k when agent k is placed at the leaf vertex incident to e_j ; if $k = i$, c_{kj} is equal to the cost of job e_j for agent k when agent k is placed at the internal vertex P_n . Compute a maximum weight matching of G' and allocate agent i all jobs matched to her. Since the weight of each edge in G' is positive, the matching is Y -saturated. Moreover, place each agent $j \neq i$, to the leaf vertex incident to the edge matched to her (if any) and then assign her the edge matched to her. Then we place agents currently having no job at unfilled positions arbitrarily. The resulting assignment is the candidate corresponding to agent i .

Since there are n options for the agent at P_n , we in total have n candidate assignments. Suppose that the assignment $(\mathcal{A}^*, \sigma^*)$ minimizes utilitarian social cost among all assignments, and moreover, agent i^* is placed at P_n in $(\mathcal{A}^*, \sigma^*)$. Note that there exists a candidate assignment $(\mathcal{A}', \sigma')$ where agent i^* is at the internal vertex P_n . Then the maximum weight matching ensures that the total cost of agents in $(\mathcal{A}', \sigma')$ is no greater than that in $(\mathcal{A}^*, \sigma^*)$, that is, $(\mathcal{A}', \sigma')$ also achieves the minimum utilitarian social cost. Therefore, by visiting n candidate assignments, we can compute the utilitarian social cost minimization assignment. Creating bipartite graph G' and computing the maximum weight matching takes time polynomial in n . $\square$

We remark that the above theorem and its proof can be extended to the case where G consists of a constant number r of star graphs. Create a candidate assignment through maximum weight matching for every $S \subseteq N$ with $|S| = r$ and every possible distribution of S to the r internal vertices. Then there are in total $\binom{n}{r} r!$ candidate assignments, and hence, the running time is polynomial in n . However, when the number of stars is not a constant, computing the utilitarian social cost minimization assignment is NP-hard, as suggested by the proof of Theorem 4, where the created job-position instance has $4n + 5m$ agents and $3n + m$ stars.

THEOREM 8. *When G is a star and agents have general cost functions, we can compute the egalitarian social cost minimization assignment in polynomial time.*

The omitted proofs can be found in the Appendix.

4 Computing MMS Fair Assignments

In this section, we focus on MMS fairness. We first study the problem of computing the MMS value with respect to a given cost function and then consider the existence of α -MMS assignments.

4.1 For MMS value

We first study the problem of computing the MMS value of a given cost function. We prove that the problem is NP-hard even for symmetric cost functions.

THEOREM 9. *For symmetric cost functions, it is NP-hard to compute the MMS value.*

PROOF. It suffices to prove the NP-completeness of the decision version: given a symmetric cost function $c(\cdot)$ and an integer W , deciding whether there exists an assignment such that the cost of every position is at most W . We derive the reduction from the PARTITION problem [19]: given a set $\{p_1, \dots, p_r\}$ of r integers whose sum is $2T$, can $[r]$ be partitioned into two indices sets I_1 and I_2 such that $\sum_{j \in I_1} p_j = \sum_{j \in I_2} p_j = T$?

We create a job-position instance where G has $4r + 2$ vertices and $6r$ edges. The vertices are described in detail as follows:

- (1) r common vertices $P_1, \dots, P_r$;
- (2) $3r$ gadget vertices such that each P_j corresponds to three gadget vertices G_j^1, G_j^2, G_j^3 ;
- (3) 2 critical vertices P_1^*, P_2^* .

Next, we create the edges and the cost. For each vertex P_j , its degree is 3 and its adjacent vertices are P_1^*, P_2^*, G_j^1 . The cost of edges (P_j, P_1^*) and (P_j, P_2^*) is p_j and the cost of (P_j, G_j^1) is $T - p_j$. For each $j \in [r]$, gadget vertices G_j^1, G_j^2, G_j^3 form a cycle of length 3, and the cost of the edge in the cycle is T . At this point, the construction of G and of the cost function is completed. Define $W = T$.

In order to ensure the cost of every position at most T , for each P_j and G_j^1, G_j^2, G_j^3 , the three edges in the cycle must be allocated such that each of G_j^1, G_j^2, G_j^3 receives one of them, resulting in each gadget position having cost exactly T . Then edge (P_j, G_j^1) , with cost $T - p_j$, must be allocated to P_j . Accordingly, position P_j can receive at most one additional edge from (P_j, P_1^*) or (P_j, P_2^*) to ensure that its cost does not exceed T .

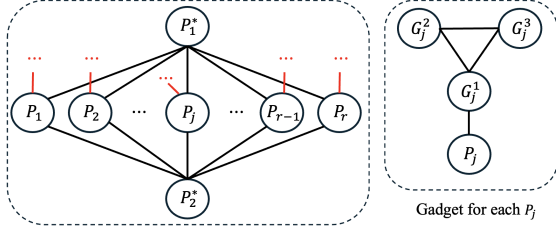


Figure 2: The illustration of the created graph in the proof of Theorem 9.

Suppose that we have a “yes” instance of PARTITION and let I_1 and I_2 be a solution. For each P_j and G_j^1, G_j^2, G_j^3 , allocate edges in the cycle to gadget vertices such that each G_j^t , with $t \in [3]$, receives one such edge and has cost T . Allocate (P_j, G_j^1) to P_j . Next, for any $j \in I_1$, allocate (P_j, P_1^*) to P_1^* and (P_j, P_2^*) to P_2^* . For any $j \in I_2$, allocate (P_j, P_1^*) to P_j and allocate (P_j, P_2^*) to post P_2^* . At this point, all edges are allocated. As $\sum_{j \in I_1} p_j = \sum_{j \in I_2} p_j = T$, it is not hard to verify that in the resulting assignment, the cost of each position is at most (indeed equal to) T .

For the other direction, suppose that there exists an assignment $\mathcal{A}$ such that the cost of each position is at most T . We drop the permutation σ as the cost of jobs is always computed by c . Next, based on $\mathcal{A}$, we construct a solution to PARTITION problem. Again in $\mathcal{A}$, for each P_j and G_j^1, G_j^2, G_j^3 , the three edges in the cycle are allocated to the gadget vertices, which constraints that edge (P_j, G_j^1) must be allocated to P_j . Hence, P_j can additionally receive at most one from (P_j, P_1^*) or (P_j, P_2^*) .

Let $N_1 = \{j \in [r] \mid (P_j, P_1^*) \text{ is assigned to } P_j \text{ in } \mathcal{A}\}$ and $N_2 = \{j \in [r] \mid (P_j, P_2^*) \text{ is assigned to } P_j \text{ in } \mathcal{A}\}$. Then we have $N_1 \cap N_2 = \emptyset$; otherwise, for each $t \in N_1 \cap N_2$, position P_t has a cost higher than T . Moreover, as $\sum_{j \in [r]} p_j = 2T$, it must be the case that $N_1 \cup N_2 = [r]$; otherwise in $\mathcal{A}$, the total cost of edges allocated to P_1^*, P_2^* exceeds $2T$, making it impossible for each critical position having cost no greater than T .

Since $N_1 \cap N_2 = \emptyset$, for any $j \in N_1$, edge (P_j, P_2^*) is allocated to P_2^* in $\mathcal{A}$, resulting in cost $\sum_{j \in N_1} p_j$ for P_2^* . Similarly, for any $j \in N_2$, edge (P_j, P_1^*) is allocated to P_1^* , yielding cost $\sum_{j \in N_2} p_j$ for P_1^* . Since each position has cost at most T in $\mathcal{A}$, we have

$$\max \left(\sum_{j \in N_1} p_j, \sum_{j \in N_2} p_j \right) \leq T,$$

which implies

$$\sum_{j \in N_1} p_j = \sum_{j \in N_2} p_j = T,$$

as $\sum_{j \in N_1 \cup N_2} p_j = \sum_{j \in [r]} p_j = 2T$. Therefore, N_1 and N_2 are a solution to the PARTITION problem. $\square$

The reduction in the proof of Theorem 9 also implies the following hardness results. When agents have symmetric and identical cost functions,

- deciding the existence of envy-free assignments is NP-complete;

- deciding the existence of PROP assignments² is NP-complete.
- computing the egalitarian social cost minimization assignment is NP-hard.

Computing the MMS value becomes tractable when the underlying (general) cost function c is binary. We present a polynomial time algorithm (see Algorithm 1) that decides, for a given integer W , whether there exists an assignment such that each position has cost at most W . In the algorithm, for given a interim (partial) assignment $\mathcal{A}$, we will create a directed auxiliary graph $D_{\mathcal{A}}$ with n vertices $\{1, \dots, n\}$, one-to-one corresponding to vertices $\{P_1, \dots, P_n\}$ in G . In $D_{\mathcal{A}}$, there exists a directed arc (i, j) from i to j if vertices P_i and P_j are adjacent in G , and edge (P_i, P_j) has cost 1 for P_i , and is allocated to P_i in $\mathcal{A}$. For vertices in $D_{\mathcal{A}}$, we also define *saturated* vertices as follows: for $\mathcal{A}$ and a given integer W , vertex i in $D_{\mathcal{A}}$ is saturated if $c^i(A_i) = W$. Moreover, vertex i is non-saturated if $c^i(A_i) < W$. Recall that c^i is the cost function for position P_i .

Informally, the algorithm maintains the invariant that the cost of each vertex is at most W in the interim assignment. Each time, it chooses an unallocated edge. In the interim assignment $\mathcal{A}$ at that moment, if one of the vertices incident to the chosen edge is non-saturated, allocate the edge to that vertex. If both incident vertices are saturated, check whether there exists a directed path in $D_{\mathcal{A}}$ from the incident vertices to a non-saturated vertex. When such a path exists in $D_{\mathcal{A}}$, for each edge (i, j) in the path, reallocate job (P_i, P_j) to P_j . After the reallocation, one of the incident vertex becomes non-saturated. Then allocate the chosen edge to that vertex and repeat the process. If no such path exists in $D_{\mathcal{A}}$, we prove that the MMS with respect to c must exceed W .

THEOREM 10. *For a general binary cost function c , one can compute the MMS value with respect to c in polynomial time.*

PROOF. It suffices to prove that for a given W , Algorithm 1 returns True if and only if MMS is at most W . If this is the case, we can compute MMS using binary search.

For the “only if” direction, the algorithm returns True when all jobs are allocated. For each job e , if it is allocated in Line 5, the vertex has cost at most W after receiving it. If it is allocated in Line 9, the vertex has cost exactly W after receiving it. Moreover, the reallocation in Line 9 can possibly increase the cost of the non-saturated vertex (only) by one. Therefore when returns True, in the resulting assignment, each vertex has cost at most W , and accordingly, the MMS with respect to c is at most W .

For the “if” direction, we prove that when the algorithm returns False, the value of MMS must be higher than W . Suppose that at some point, the interim assignment is $\mathcal{A} = (A_1, \dots, A_n)$ and the algorithm chooses the unallocated job e' with incident positions P_i and P_j . There is no directed path in $D_{\mathcal{A}}$ from i or j to a non-saturated vertex. Let $S_i \subseteq [n]$ and $S_j \subseteq [n]$ be respectively the set of vertices reachable from i and j in $D_{\mathcal{A}}$; to avoid confusion, we let $i \notin S_i$ and $j \notin S_j$. Below, we say a job is *unavoidable* if the cost of the job is 1 for both of its incident positions.

We first consider the case when $j \in S_i$, and hence $S_j \subseteq S_i$. For each $k \in S_i$, vertex k is saturated. Then $c^k(A_k) = W$ and moreover

²For an instance where n agents have symmetric cost functions for jobs M , an assignment is proportional (PROP) if each agent i receives cost at most $\frac{1}{n} c_i(M)$.

Algorithm 1 Computing MMS for binary cost functions

Input: A job-position instance with graph G , binary function c , and an integer W .

Output: True or False.

- 1: For each edge, if it has cost zero for one of its incident vertices, then assign the edge to that vertex. Let E_0 be the set of unallocated edges and $\mathcal{A}$ be the current assignment where A_i refers to the edges allocated to P_i .
- 2: **while** $E_0 \neq \emptyset$ **do**
- 3: Arbitrarily choose $e \in E_0$ and let P_i, P_j be the two vertices incident to e .
- 4: **if** $c^i(A_i) < W$ **or** $c^j(A_j) < W$ **then**
- 5: Allocate e to the vertex with cost less than W . Update $\mathcal{A}$ and $E_0 \leftarrow E_0 \setminus \{e\}$.
- 6: **else if** there is no directed path in $D_{\mathcal{A}}$ from vertices i or j to a non-saturated vertex **then**
- 7: Return False and terminate.
- 8: **else if** there exists a directed path in $D_{\mathcal{A}}$ from vertices i or j and a non-saturated vertex **then**
- 9: Suppose that the directed path is $i = i_0 \rightarrow i_1 \rightarrow \dots \rightarrow i_{r-1} \dots \rightarrow i_r$. Starting from $\mathcal{A}$, for each $s \in \{0, 1, \dots, r-1\}$, reallocate job $(P_{i_s}, P_{i_{s+1}})$ to $P_{i_{s+1}}$. Moreover, allocate the chosen job e to P_i , and update $\mathcal{A}$ and $E \leftarrow E \setminus \{e\}$.
- 10: **end if**
- 11: **end while**
- 12: Return True.

A_k contains W unavoidable jobs. Note that if a job $e \in A_k$ has cost 1 for P_k but has cost 0 for the other incident vertex, denoted as $P_{k'}$, job e is allocated to $P_{k'}$ in Line 1 and never gets reallocated. For any $k \in S_i$ and $\ell \notin S_i$ (if any), if P_k and P_ℓ are adjacent in G and the cost of (P_k, P_ℓ) for P_k is one, then job (P_k, P_ℓ) is allocated to P_ℓ in $\mathcal{A}$. Otherwise, there is a directed arc from k to ℓ in $D_{\mathcal{A}}$, making $\ell \in S_i$, a contradiction. We now consider positions $\{P_f\}_{f \in S_i \cup \{i\}}$. Since the chosen job e' is also unavoidable and $j \in S_i$, there are $W(|S_i| + 1) + 1$ unavoidable jobs, and for each of them, both incident positions are in $\{P_f\}_{f \in S_i \cup \{i\}}$. By the pigeonhole principle, in any assignment, there exists at least one position in $\{P_f\}_{f \in S_i \cup \{i\}}$ having cost at least $W + 1$. Therefore, the MMS is at least $W + 1$.

For the case where $j \notin S_i$, by arguments similar to the above, we can claim that there are $W(|S_i| + |S_j| + 2) + 1$ unavoidable jobs, and for each of them, both incident positions are in $\{P_f\}_{f \in S_i \cup S_j \cup \{i, j\}}$. Therefore in any assignment, there exists at least one position in $\{P_f\}_{f \in S_i \cup S_j \cup \{i, j\}}$ having cost at least $W + 1$, which implies that the MMS is at least $W + 1$.

The number of jobs is at most $n(n-1)$ and checking the existence of the directed path in Line 6 takes time at most $O(n^2)$ by DFS. The running time of Algorithm 1 is at most $O(n^4)$. Using binary search, one can compute the value of MMS in time polynomial in n . $\square$

When G is restricted to a path, cycle, or star, one can compute the MMS values for general cost functions. Below, we present the result for cycles, and defer the results for paths and stars to the subsequent section.

THEOREM 11. *For cycles and general cost function c , we can compute the MMS value with respect to c in polynomial time.*

4.2 For Approximate MMS Assignments

We are now concerned with the existence of α -MMS assignment. Unfortunately, there is no assignment with a bounded approximation guarantee, even for general binary cost functions.

THEOREM 12. *For general binary cost functions and any $\beta \geq 1$, there exists an instance with three agents where no assignment is β -MMS, even when G is a cycle of length 3.*

PROOF. Consider an instance with three agents where graph G has three vertices P_1, P_2, P_3 and three edges, forming a cycle. For any $i, j \in [3]$ and $i < j$, denote by $e_{i,j}$ the edge incident to P_i and P_j , i.e., three edges are $e_{1,2}, e_{1,3}, e_{2,3}$. For each agent $i \in [2]$, her cost function is

- (1) when at P_1 , $c_1^1(e_{1,2}) = 1$ and $c_1^1(e_{1,3}) = 0$;
- (2) when at P_2 , $c_2^2(e_{1,2}) = 0$ and $c_2^2(e_{2,3}) = 1$;
- (3) when at P_3 , $c_3^3(e_{1,3}) = 1$ and $c_3^3(e_{2,3}) = 0$.

For agent 3, her cost function is: for each P_j and each job e incident to P_j , $c_3^j(e) = 1 - c_1^j(e)$. Informally, agents 1 and 2 have cost 0 when assigning edges to positions in a clockwise manner, while agent 3 has cost 0 in the allocation is in an anti-clockwise manner. It is easy to check that $\text{MMS}_i = 0$ for every $i \in [3]$. Thus, if an assignment is β -MMS, the cost of each agent must be 0 in that assignment.

For the sake of contradiction, suppose that assignment $(\mathcal{A}, \sigma)$ is β -MMS and thus each agent i has cost 0. Suppose that $e_{1,2}$ is allocated to P_2 , then $e_{2,3}$ (resp., $e_{1,3}$) must be allocated to P_3 (resp., P_1); otherwise, some vertex receives both incident edges, resulting cost 1. Then in $(\mathcal{A}, \sigma)$, agent 3 has cost 1, a contradiction. For the case where $e_{1,2}$ is allocated to P_1 in $\mathcal{A}$, edges $e_{1,3}$ and $e_{2,3}$ must be allocated to P_3 and P_2 respectively. Accordingly, both agents 1 and 2 have cost 1 in $(\mathcal{A}, \sigma)$, a contradiction. Therefore, no assignment has a finite approximation guarantee in the created job-position instance. $\square$

We next show that when cost functions are symmetric, there always exist 2-MMS assignments. First, we introduce a reduction property on the value of MMS. That is, if we remove a position and all edges incident to that position, the maximin share value does not increase. Formally, let $\text{MMS}(n, G, c)$ be the maximin share with respect to cost function c and graph G with n vertices. Let G' be the reduced graph obtained by removing a vertex and all its incident edges from G , and $\text{MMS}(n-1, G', c)$ be the corresponding maximin share. We claim that $\text{MMS}(n-1, G', c) \leq \text{MMS}(n, G, c)$.

LEMMA 13. *Suppose that c is a general cost function defined on job-position instance with graph G having n vertices. Let G' be the graph after the removal of a vertex and all edges incident to that vertex from G . Then $\text{MMS}(n-1, G', c) \leq \text{MMS}(n, G, c)$.*

PROOF. Let $P_1, \dots, P_n$ be the n positions in G . Suppose that assignment $\mathcal{A} = (A_1, \dots, A_n)$ defines $\text{MMS}(n, G, c)$, that is, each position P_j receives A_j and has cost $c^j(A_j) \leq \text{MMS}(n, G, c)$ where c^j is the cost function for P_j . Without loss of generality, assume that G' is obtained by removing P_n and all edges incident to P_n , denoted as S . Then for each $j \in [n-1]$, define $B_j = A_j \setminus S$. Hence, $\{B_1, \dots, B_{n-1}\}$ is an $(n-1)$ -partition of all edges in G' , and moreover, edges in B_j are incident to P_j . Consider the assignment $\mathcal{B} = (B_1, \dots, B_{n-1})$

where B_j is allocated to position P_j for all $j \in [n - 1]$. By the definition of $\text{MMS}(n - 1, G', c)$, we have

$$\begin{aligned} \text{MMS}(n - 1, G', c) &\leq \max_{j \in [n-1]} c^j(B_j) \\ &\leq \max_{j \in [n]} c^j(A_j) = \text{MMS}(n, G, c), \end{aligned}$$

completing the proof. $\square$

THEOREM 14. *For symmetric cost functions, there always exist 2-MMS assignments.*

PROOF. Recall that R_j refers to the set of edges incident to P_j for every position P_j . If P_j and $P_{j'}$ are adjacent in G , then $R_j \cap R_{j'} \neq \emptyset$. For any agent i , by the definition of MMS_i , we have $\text{MMS}_i \geq \frac{1}{n} c_i(E)$. By counting the edges, for each agent i , it holds that

$$\sum_{j \in [n]} \sum_{e \in R_j} c_i(e) = 2 \sum_{e \in E} c_i(e) \leq 2n \text{MMS}_i.$$

By pigeonhole principle, for each agent i , there exists a position such that agent i has cost no greater than 2MMS_i when receiving all edges incident to that position. We name such a position as *critical position* for that agent.

Consider the following procedure. First, we arbitrarily choose an agent. Place the chosen agent at her critical position and allocate her all incident edges. Then remove the chosen agent, the filled position, and all its incident edges from G , resulting in another instance with graph G^1 . Next, we again arbitrarily choose a remaining agent and place her at her critical position corresponding to G^1 . Note the maximin share of an agent in G^1 is defined by an $(n - 1)$ -partition. Allocate the chosen agent all edges incident to her, and remove the agent, the position, and allocated edges. Repeat this process until $n - 1$ agents are removed, where all edges of G are also allocated. In the resulting assignment, by repeatedly using Lemma 13, each chosen agent has cost at most 2MMS_i where MMS_i is defined on G . The last agent does not receive an edge and has cost 0, which completes the proof. $\square$

In the following, we focus on a path graph and a star graph, where MMS assignments always exist and can be computed in polynomial time, proved in the appendix.

THEOREM 15. *When G is a star, the MMS value of a general cost function can be computed in polynomial time. Moreover when agents have general cost functions, the MMS assignment always exists and can be computed in polynomial time.*

PROOF. Computing MMS value is equivalent to the problem of computing the minimum egalitarian social cost when agents have identical cost functions. Thus, by Theorem 8, for a star graph, the MMS value of a general cost function can be computed in polynomial time.

Next, we study the MMS assignment and consider the following algorithm.

- Step 1: While there exists an agent i , a leaf vertex P_j with incident edge e , such that $c_i^j(e) \leq \text{MMS}_i$, allocate e to P_j and place agent i at P_j . If all edges are allocated, place the last agent to the internal vertex and terminate; otherwise, goes to Step 2.

- Step 2: Assign all unallocated edges to the internal vertex, and arbitrarily place all agents who have not been distributed to vertices at unfilled vertices.

If the algorithm terminates at Step 1, then clearly the resulting assignment is MMS. When the algorithm terminates at Step 2, we will show that the agent (say agent n) placed at the internal vertex P_n satisfies MMS. Denote by E' the set of edges allocated in Step 1. We now consider the moment right before Step 2. By the condition of while-loop in Step 1, for any $e \in E \setminus E'$ and any (currently) unfilled position P_j with $j \neq n$, $c_n^j(e) > \text{MMS}_n$ holds; otherwise, agent n is chosen in Step 1. Thus, in the n -partition that defines MMS_n , each job $e \in E \setminus E'$ must be allocated to internal vertex P_n , which implies $\sum_{e \in E \setminus E'} c_n^n(e) \leq \text{MMS}_n$. Therefore, the resulting assignment is MMS, and the running time is at most $O(n^2)$. $\square$

Indeed the algorithm in the above proof can also be used to compute the MMS value. (By replacing MMS_i with an integer W in Step 1, we can decide in polynomial whether the MMS value of the function c is at most W or not.) We also remark that the above theorem and the proof can be extended to the case where G has r components, each being a star. In particular, we implement Step 1 for every component and then implement Step 2 for the component where there exist unallocated edges.

THEOREM 16. *When G is a path, the MMS value of a general cost function can be computed in polynomial time. Moreover when agents have general cost functions, the MMS assignment always exists and can be computed in polynomial time.*

We remark that the above theorem can be extended to the case where G consists of r components, each being a path.

5 Conclusion

In this paper, we introduce the problem of fair allocation with position-aware valuations. We provide a complete set of results when the constraints form a graph. In particular, our results identify the tractable and intractable cases of computing efficient assignments and the MMS values. We also design a polynomial-time algorithm that computes a 2-approximate MMS assignment for symmetric cost functions. It is interesting to improve the approximation ratio in future work.

Beyond efficiency and MMS, we also consider the notion of envy-free up to one item (EF1), a relaxation of envy-freeness. This notion requires that the envy can be eliminated by removing a single item from the envied agent's bundle. We note that although EF1 allocations always exist in the canonical setting with monotone valuations [9, 35], this fairness notion is not guaranteed in our context, and the problem of deciding the existence of an EF1 assignment is NP-complete. We defer the discussion of EF1 to Appendix C.

Our new framework opens up many intriguing research avenues. For instance, one could explore hypergraphs, where edges can connect more than two vertices, as well as other types of constraints, including those between agents and positions. It would be interesting to investigate alternative fairness notions, such as proportionality and its relaxations, as well as concepts like price share fairness. Finally, the complementary problem of allocating goods also presents a compelling direction for further study.

References

- [1] Mahyar Afshinmehri, Alireza Danaei, Mehrafarin Kazemi, Kurt Mehlhorn, and Nidhi Rathi. 2025. EFX Allocations and Orientations on Bipartite Multi-graphs: A Complete Picture. In *AAMAS. International Foundation for Autonomous Agents and Multiagent Systems / ACM*, 32–40.
- [2] Hannaneh Akrami and Jugal Garg. 2024. Breaking the 3/4 Barrier for Approximate Maximin Share. In *SODA. SIAM*, 74–91.
- [3] Hannaneh Akrami, Kurt Mehlhorn, Masoud Seddighin, and Golnoosh Shahkarami. 2023. Randomized and Deterministic Maximin-share Approximations for Fractionally Subadditive Valuations. In *NeurIPS*.
- [4] Georgios Amanatidis, Haris Aziz, Georgios Birmpas, Aris Filos-Ratsikas, Bo Li, Hervé Moulin, Alexandros A. Voudouris, and Xiaowei Wu. 2023. Fair division of indivisible goods: Recent progress and open questions. *Artif. Intell.* 322 (2023), 103965.
- [5] Georgios Amanatidis, Aris Filos-Ratsikas, and Alkmini Sgouritsa. 2024. Pushing the Frontier on Approximate EFX Allocations. In *EC. ACM*, 1268–1286.
- [6] Siddharth Barman, Arindam Khan, Sudarshan Shyam, and K. V. N. Sreenivas. 2023. Guaranteeing Envy-Freeness under Generalized Assignment Constraints. In *EC. ACM*, 242–269.
- [7] Siddharth Barman and Sanath Kumar Krishnamurthy. 2020. Approximation Algorithms for Maximin Fair Division. *ACM Trans. Economics and Comput.* 8, 1 (2020), 5:1–5:28.
- [8] Piotr Berman, Marek Karpinski, and Alex D. Scott. 2003. Approximation Hardness of Short Symmetric Instances of MAX-3SAT. *Electron. Colloquium Comput. Complex.* TR03-049 (2003).
- [9] Umang Bhaskar, A. R. Sricharan, and Rohit Vaish. 2021. On Approximate Envy-Freeness for Indivisible Chores and Mixed Resources. In *APPROX/RANDOM (LIPIcs, Vol. 207)*. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 1:1–1:23.
- [10] Vittorio Bilò, Ioannis Caragiannis, Michele Flammini, Ayumi Igarashi, Gianpiero Monaco, Dominik Peters, Cosimo Vinci, and William S. Zwicker. 2022. Almost envy-free allocations with connected bundles. *Games Econ. Behav.* 131 (2022), 197–221.
- [11] Arpita Biswas and Siddharth Barman. 2018. Fair Division Under Cardinality Constraints. In *IJCAI. ijcai.org*, 91–97.
- [12] Sylvain Bouveret, Katarína Cechlárová, Edith Elkind, Ayumi Igarashi, and Dominik Peters. 2017. Fair Division of a Graph. In *IJCAI. ijcai.org*, 135–141.
- [13] Eric Budish. 2011. The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes. *J. Political Econ.* 119, 6 (2011), 1061–1103.
- [14] Ioannis Caragiannis, David Kurokawa, Hervé Moulin, Ariel D. Procaccia, Nisarg Shah, and Junxing Wang. 2019. The Unreasonable Fairness of Maximum Nash Welfare. *ACM Trans. Economics and Comput.* 7, 3 (2019), 12:1–12:32.
- [15] George Christodoulou, Amos Fiat, Elias Koutsoupias, and Alkmini Sgouritsa. 2023. Fair allocation in graphs. In *EC. ACM*, 473–488.
- [16] George Christodoulou and Symeon Mastrakoulis. 2025. Exact and approximate maximin share allocations in multi-graphs. *CoRR abs/2506.20317* (2025).
- [17] Argyrios Deligkas, Eduard Eiben, Tiger-Lily Goldsmith, and Viktoriia Korchemna. 2025. EF1 and EFX Orientations. In *IJCAI. ijcai.org*, 56–63.
- [18] Edith Elkind, Ayumi Igarashi, and Nicholas Teh. 2024. Fair Division of Chores with Budget Constraints. In *SAGT (Lecture Notes in Computer Science, Vol. 15156)*. Springer, 55–71.
- [19] M. R. Garey and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman.
- [20] Mohammad Ghodsi, Mohammad Taghi Hajiaghayi, Masoud Seddighin, Saeed Seddighin, and Hadi Yami. 2018. Fair Allocation of Indivisible Goods: Improvements and Generalizations. In *EC. ACM*, 539–556.
- [21] Mohammad Ghodsi, Mohammad Taghi Hajiaghayi, Masoud Seddighin, Saeed Seddighin, and Hadi Yami. 2022. Fair allocation of indivisible goods: Beyond additive valuations. *Artif. Intell.* 303 (2022), 103633.
- [22] Kevin Hsu. 2025. EFX Orientations of Multigraphs. In *ECAI (Frontiers in Artificial Intelligence and Applications)*. IOS Press, 3783–3790.
- [23] Xin Huang and Pinyan Lu. 2021. An Algorithmic Framework for Approximating Maximin Share Allocation of Chores. In *EC. ACM*, 630–631.
- [24] Xin Huang and Erel Segal-Halevi. 2023. A Reduction from Chores Allocation to Job Scheduling. In *EC. ACM*, 908.
- [25] Halvard Hummel. 2025. Maximin Shares in Hereditary Set Systems. *ACM Trans. Economics and Comput.* 13, 3 (2025), 12:1–12:33.
- [26] Halvard Hummel and Magnus Lie Hetland. 2022. Fair allocation of conflicting items. *Auton. Agents Multi Agent Syst.* 36, 1 (2022), 8.
- [27] Halvard Hummel and Magnus Lie Hetland. 2022. Maximin Shares Under Cardinality Constraints. In *EUMAS (Lecture Notes in Computer Science, Vol. 13442)*. Springer, 188–206.
- [28] Ayumi Igarashi. 2023. How to Cut a Discrete Cake Fairly. In *AAAI. AAAI Press*, 5681–5688.
- [29] Alireza Kaviani, Alireza Keshavarz, Masoud Seddighin, and Amir Mohammad Shahrezaei. 2025. Improved Approximate EFX Guarantees for Multigraphs. *CoRR abs/2506.09288* (2025).
- [30] Yatharth Kumar, Sarfaraz Equbal, Rohit Gurjar, Swaprava Nath, and Rohit Vaish. 2024. Fair Scheduling of Indivisible Chores. In *AAMAS. International Foundation for Autonomous Agents and Multiagent Systems / ACM*, 2345–2347.
- [31] David Kurokawa, Ariel D. Procaccia, and Junxing Wang. 2018. Fair Enough: Guaranteeing Approximate Maximin Shares. *J. ACM* 65, 2 (2018), 8:1–8:27.
- [32] Bo Li, Minming Li, and Ruilong Zhang. 2021. Fair Scheduling for Time-dependent Resources. In *NeurIPS*. 21744–21756.
- [33] Bo Li, Fangxiao Wang, and Yu Zhou. 2023. Fair Allocation of Indivisible Chores: Beyond Additive Costs. In *NeurIPS*.
- [34] Zhentao Li and Adrian Vetta. 2021. The Fair Division of Hereditary Set Systems. *ACM Trans. Economics and Comput.* 9, 2 (2021), 12:1–12:19.
- [35] Richard J. Lipton, Evangelos Markakis, Elchanan Mossel, and Amin Saberi. 2004. On approximately fair allocations of indivisible goods. In *EC. ACM*, 125–131.
- [36] Hervé Moulin. 2019. Fair division in the internet age. *Annu. Rev. Econ.* 11, 1 (2019), 407–441.
- [37] Warut Suksompong. 2021. Constraints in fair division. *SIGames Exch.* 19, 2 (2021), 46–61.
- [38] Xiaowei Wu, Bo Li, and Jiarui Gan. 2025. Approximate envy-freeness in indivisible resource allocation with budget constraints. *Inf. Comput.* (2025), 105264.
- [39] Ryo Yoshinaka. 2005. Higher-Order Matching in the Linear Lambda Calculus in the Absence of Constants Is NP-Complete. In *RTA (Lecture Notes in Computer Science, Vol. 3467)*. Springer, 235–249.
- [40] Jinghan A. Zeng and Ruta Mehta. 2025. On the Structure of EFX Orientations on Graphs. In *AAMAS. International Foundation for Autonomous Agents and Multiagent Systems / ACM*, 2309–2316.
- [41] Yu Zhou, Tianze Wei, Minming Li, and Bo Li. 2024. A Complete Landscape of EFX Allocations on Graphs: Goods, Chores and Mixed Manna. In *IJCAI. ijcai.org*, 3049–3056.

Technical Appendices and Supplementary Material

A Missing Proofs in Section 3

A.1 Proof of Theorem 8

PROOF OF THEOREM 8. For a star graph, i.e., $G = K_{1,n-1}$, without loss of generality, let P_n be the internal vertex and $\{P_j\}_{j \in [n-1]}$ be the set of leaves. We will prove that for any $i \in [n]$ and an integer W , we can decide in polynomial time whether there exists an assignment such that agent i is at position P_n and its egalitarian social cost is no greater than W . Moreover, we can compute such an assignment when it exists. Let us now assume that these two claims are true. Then by binary search, we can compute the assignment with minimum egalitarian social cost among all assignments where agent i is placed at P_n . That is, we compute the candidate assignment corresponding to agent i . By visiting n candidate assignments, we find the assignment that minimizes egalitarian social cost.

We now prove the two claims mentioned above. Fix agent i and an integer W . Consider the following algorithm.

- (1) Create a weighted bipartite graph G' with two parts of vertices X and Y where for each agent $h \in [n] \setminus \{i\}$, create a vertex x_h in X and for each job e_j , create a vertex y_j in Y . For any h and j , if the cost of job e_j for agent h is at most W when agent h is placed at the leaf vertex incident to e_j , create edge (x_h, y_j) and let it weight be $c_i^n(e_j)$.
- (2) Compute the maximum weight matching μ of G' where agents in N' and jobs in E' is matched.
- (3) If $c_i^n(E \setminus E') \leq W$, return True; otherwise, return False.

Next, we show that the algorithm returns True if and only if there exists an assignment such that agent i is placed at P_n and its egalitarian social cost is at most W . For the “only if” direction, place agents in N' at leaf vertices in a way such that each agent in N' can receive their matched job in μ . Then, allocate each agent in N' the job matched to her in μ , and allocate all remaining jobs to agent i at position P_n . Since the algorithm returns True, the resulting assignment has egalitarian social cost no greater than W .

For the “if” direction, suppose that $(\mathcal{A}, \sigma)$ is an assignment where agent i is at P_n and its egalitarian social cost at most W . Let $S \subseteq [n] \setminus \{i\}$ be the set of agents, each of whom receives a job in $(\mathcal{A}, \sigma)$. Note that agents S are placed at leaf vertices in $(\mathcal{A}, \sigma)$. Without loss of generality, assume $A_{\sigma(j)} = \{e_j\}$ for every $j \in S$. Thus, in bipartite graph G' created in the algorithm, edge (x_j, y_j) exists for every $j \in S$. Given that edges $\{(x_j, y_j)\}_{j \in S}$ form a matching in G' , it holds that

$$\sum_{e \in E'} c_i^n(e) \geq \sum_{j \in S} c_i^n(e_j),$$

which implies

$$\sum_{e \in E \setminus E'} c_i^n(e) \leq \sum_{e \in E \setminus \bigcup_{j \in S} e_j} c_i^n(e) = c_i^n(A_n) \leq W.$$

Therefore, the algorithm returns True. $\square$

B Missing Proofs in Section 4

B.1 Proof of Theorem 11

PROOF OF THEOREM 11. It suffices to prove that given an integer W , we can decide in polynomial time whether there exists an

assignment such that the cost of each position is no greater than W . First, we arbitrarily choose an edge and label it e_1 . Then, in a clockwise direction, label other edges $e_2, \dots, e_n$, and hence, e_1, e_n are adjacent. Consider the following algorithm.

- Step 1: Visit edges clock-wisely from e_1 to e_n . If there exists an edge such that allocating it to any of its incident vertices results in a cost of higher than W , return False. Let E^* be a subset of edges such that each $e \in E^*$ can only be allocated to one of its incident vertices, denoted as $P_{f(e)}$, without exceeding cost W . If $E^* = \emptyset$, return True.
- Step 2: For each $e \in E^*$, assign e to $P_{f(e)}$. If in the partial allocation, there exists a vertex with cost higher than W , then return False. Moreover, if in the partial allocation, all allocated edges are assigned either to the incident vertices on their right or to those on their left, then return True.
- Step 3: Let $e^* \in E^*$ be an edge such that $P_{f(e^*)}$ is the incident vertex to right of e^* . Starting from the edge adjacent to e^* and to the right of e^* , proceed unallocated edges in a clockwise direction. For the underlying unallocated edge e , if it can be assigned to the incident vertex on its left without exceeding cost W , then assign it to that vertex. Otherwise, assign e to the incident vertex on its right, and if the cost of that vertex exceeds W , return False. Repeat Step 3 until, in some round, the algorithm returns False, or all edges are allocated. If the latter happens, return True.

We prove that the algorithm returns True if and only if the MMS value is no greater than W . For the “only if” direction, when the algorithm returns True at Step 1, we create an assignment where each vertex receives one edge. When the algorithm returns True at Step 2 and each $e \in E^*$ is assigned to the incident vertex on its right (resp., left), allocate each unallocated edge to the incident vertex on its right (resp., left). When the algorithm returns True at Step 3, we consider the resulting allocation. In these three cases, each vertex in the allocation that we consider has cost no greater than W , which implies that the MMS is at most W .

For the “if” direction, we prove that if the algorithm returns False, then in every assignment, there exists at least one vertex having cost higher than W , which is equivalent to that the MMS value exceeds W . If the algorithm returns False in Step 1, then there exists a job e such that allocating e to either of its incident vertices results in cost higher than W . If the algorithm returns False in Step 2, there exists two jobs e, e' such that in order to make the cost no greater than W , they must be allocated to the vertex incident to both of them. However, the vertex receiving both e, e' has cost higher than W . Therefore, in these two cases, there is no assignment such that every vertex has cost at most W .

For the case where the algorithm returns False at Step 3, towards a contradiction, assume that there exists assignment $\mathcal{A}$ where each vertex has cost at most W . First, e^* is guaranteed to exist; otherwise, the algorithm returns True in Step 2. For each $e \in E^*$, we call e is a *compulsory* edge or is *compulsory*. By the algorithm, each $e \in E^*$ must be allocated to $P_{f(e)}$ in $\mathcal{A}$.

Suppose that the algorithm returns False when it proceeds job e' and the partial assignment at that moment is $\mathcal{B}$. Let $P_{l'}$ and $P_{r'}$ be the vertices incident to e' and on its left and right respectively. In $\mathcal{B}$, both $P_{l'}, P_{r'}$ receive edges, and moreover, allocating e' to any

of them results in cost exceeding W . Note that $P_{l'}$ must receive a compulsory edge in $\mathcal{B}$ as the allocation in Step 3 proceeds edges in a clockwise direction. Thus in $\mathcal{A}$, e' must be allocated to $P_{l'}$.

If $P_{l'}$ receives a compulsory edge in $\mathcal{B}$, then in $\mathcal{A}$, vertex $P_{l'}$ must receive both of its incident and has cost higher than W because when proceeding e' in Step 3, it cannot be allocated to $P_{l'}$. This contradicts the definition of $\mathcal{A}$. Hence in $\mathcal{B}$, $P_{l'}$ must receive a non-compulsory edge, denoted as e'' , and moreover, e'' is to the left of $P_{l'}$. Now in $\mathcal{A}$, as e' is allocated to $P_{l'}$, edge e'' must be allocated to $P_{l''}$ in $\mathcal{A}$ because e', e'' cannot be assigned to $P_{l'}$ together based on Step 3 of the algorithm.

We repeat this process and will eventually reach some edge e such that its adjacent edge on the left is compulsory; recall that e^* is a compulsory edge and the process start from e^* . Then in $\mathcal{A}$, except for e , the two vertices incident to e also receive an edge each. However, based on Step 3 of the algorithm, each vertex incident to e has cost higher than W if it receives two of its incident edges. Therefore in $\mathcal{A}$, some vertex must have cost higher than W , a contradiction.

The algorithm takes time at most $O(n)$, and using binary search, we can compute the MMS value in time polynomial in n . $\square$

B.2 Proof of Theorem 16

PROOF OF THEOREM 16. We first prove the existence results. Let us now assume that MMS_i 's are known and we will prove later that each MMS_i can be computed in polynomial time. Consider the following algorithm.

- While there exists unallocated edges, find an agent $i \in N$ and a vertex P_j with the set of all (unallocated) incident edges $S_j \subseteq E$ such that $c_i^j(S_j) \leq \text{MMS}_i$. Place agent i at P_j and allocate S_j to P_j . Update $N \leftarrow N \setminus \{i\}$ and $E \leftarrow E \setminus S_j$. When all edges are allocated, arbitrarily distribute agents in N to unfilled vertices.

We prove that in each round of while-loop, there always exists an agent and a vertex meeting the condition. For each $t = 1, 2, \dots$, let G^t be the graph obtained by removing the allocated edges and the filled vertices in all rounds $0, 1, 2, \dots, t-1$. We now fix t and let N^t be the set of agents who have not been placed at vertices at the start of round t . By induction and repeatedly using Lemma 13, we can claim that for each agent $i \in N^t$,

$$\text{MMS}_i(n-t+1, G^t, c_i) \leq \text{MMS}_i(n, G, c_i).$$

For simple notations, we use $\text{MMS}_i(G^t)$ to refer to $\text{MMS}_i(n-t+1, G^t, c_i)$. We prove that for each $i \in N^t$, there exists a vertex P_k with S_k being the set of all edges incident to P_k in G^t such that $c_i^k(S_k) \leq \text{MMS}_i(G^t)$. Note that G^t can consist of multiple components, each of which is a path. Let C be an arbitrary component of G^t and suppose that C has n_C vertices. By the definition of MMS_i , it is not hard to see that $\text{MMS}_i(n_C, C, c_i) \leq \text{MMS}_i(G^t)$. We claim that in C , there exists a vertex P_k and its incident edges S_k such that $c_i^k(S_k) \leq \text{MMS}_i(n_C, C, c_i)$.

To see the claim, we can assume that in C , every vertex with degree 2 does not satisfy the condition; otherwise, the claim holds. Then in the assignment $\mathcal{A}^*$ that defines $\text{MMS}_i(n_C, C, c_i)$, each vertex in C receives at most one edge. Moreover in assignment $\mathcal{A}^*$, if the left-most edge is allocated to the left-most vertex, then we find the desired vertex in C . On the other hand, if the left-most

edge is allocated to the second left-most vertex, then by repeatedly utilizing the fact that no vertex receives two edges in $\mathcal{A}^*$, the right-most edge must be allocated to the right-most vertex. Thus, the right-most vertex in C satisfies condition. Thus, there always exists a vertex and an agent satisfying the condition in the while-loop. Therefore, the algorithm returns an MMS assignment.

Next, we show how to use the algorithm to compute MMS_i 's. For each $i \in [n]$, we implement the algorithm on the instance where n agents have identical cost function c_i . We prove that for this instance and an integer W , the above algorithm, with modifications, can decide whether there exists an assignment such that the cost of every vertex is at most W . We replace the MMS_i by W , and if all edges are allocated, let the algorithm return True; otherwise, returns False.

We prove that the (modified) algorithm returns True if and only if there exists an assignment where each vertex has cost at most W . The “only if” direction holds trivially as when returning True, every position in the resulting assignment has cost at most W . For the “if” direction, we show that if the algorithm returns False, no assignment ensures the cost of each vertex at most W . Suppose that at round $t \geq 1$, the algorithm returns False. Similarly, let G^t be the graph obtained by removing the allocated edges and the filled vertices in all rounds $0, 1, 2, \dots, t-1$. Note that G^t possibly has multiple components. Let C be an arbitrary component of G^t and suppose that C has n_C vertices. As the algorithm returns False, for each position P_j in C , the total cost of its incident edges in G^t exceeds W . Thus, assigning the left-most (resp., right-most) edge to the left-most (resp., right-most) vertex results in cost larger than W . Then in the assignment (if any) where the cost of each vertex is at most W , when restricted to edges in C , the $n_C - 1$ edges must be assigned to the $n_C - 2$ vertices, excluding the left-most and the right-most vertices. Then there must exist a vertex in C such that the total cost of its incident edges is at most W , and the algorithm would not return False in that round, a contradiction.

The algorithm takes time at most $O(n^2)$. Using binary search, one can compute MMS_i 's in time polynomial in n . $\square$

C A Discussion on EF1 Fairness

In this section, we briefly discuss our results on EF1 fairness.

DEFINITION 17 (EF). An assignment $(\mathcal{A}, \sigma)$ is envy-free (EF) if for any $i, j \in [n]$,

$$c_i^{\sigma(i)}(A_{\sigma(i)}) \leq c_i^{\sigma(j)}(A_{\sigma(j)}).$$

DEFINITION 18 (EF1). An assignment $(\mathcal{A}, \sigma)$ is envy-free up to one job (EF1) if for any i, j , there exists a job $e \in A_{\sigma(i)}$ such that $c_i^{\sigma(i)}(A_{\sigma(i)} \setminus \{e\}) \leq c_i^{\sigma(j)}(A_{\sigma(j)})$.

We show below that EF1 assignments do not always exist, and deciding their existence for a given instance is NP-complete.

THEOREM 19. When $n \geq 5$, EF1 allocations do not always exist even for identical, symmetric, and binary cost functions.

PROOF. Consider graph G that consists of a special vertex P_n and other $n-1$ vertices forming a complete graph K_{n-1} . There is one edge between P_n and K_{n-1} . The cost of the edge incident to P_n is 0 and the cost of other edges is 1. In every assignment, among

agents placed at the vertices of K_{n-1} , there exists at least one agent having a cost of at least 2. Then, when compared to the agent at P_n , the agent having cost at least 2 violates EF1. $\square$

THEOREM 20. *For symmetric binary cost functions, it is NP-complete to decide whether there exists an EF1 assignment or not.*

PROOF. These decision problems are in NP as given an assignment, one can decide whether it is EF1 or EFX in polynomial time. Below we prove for EF1 and the same proof works for EFX.

We derive a reduction from 2P2N-3SAT and create a job-position instance where G has $6n + 13m + 2$ vertices and $6n + 24m + 1$ edges. For each variable x_i , create two agents $x_i, \bar{x}_i$ and two vertices $P_{x_i}, P_{\bar{x}_i}$, corresponding to $x_i, \bar{x}_i$, respectively. For each clause C_j , let C_j^1, C_j^2, C_j^3 be respectively the first, second and third literals of C_j . Then for C_j , we create three agents C_j^1, C_j^2, C_j^3 and three vertices $P_{C_j^1}^1, P_{C_j^2}^2, P_{C_j^3}^3$ corresponding to these agents. There are also $4n + 10m + 2$ dummy agents. The detailed construction of G is presented as follows:

- For each x_i , create a gadget with 6 vertices: $P_{x_i}, P_{\bar{x}_i}$ (corresponding to agents $x_i, \bar{x}_i$) and $P_{x_i}^{dt}$ for $t \in [4]$. Vertices $P_{x_i}^{dt}$'s form a clique, i.e., there is an edge between any two of them.
- For each C_j , create a gadget with 13 vertices: for any $t \in [3], k \in [3]$, create vertex $P_{C_j}^{t,k}$ and then for any $t \in [4]$

create vertex $P_{C_j}^{dt}$. Vertices $P_{C_j}^{dt}$'s form a clique. In addition, the three vertices corresponding to each literal in C_j with the vertex corresponding to the variable from a clique. For example, if the first literal of C_j is x_i , then the 4 vertices, $P_{C_j}^{1,k}$'s and P_{x_i} , form a clique.

- Create two vertices P_0^1, P_0^2 and create edge (P_0^1, P_0^2) .

We have completed the construction of G . Next, we present the (symmetric) cost functions of agents.

- For any $i \in [n]$, agent x_i (resp., $\bar{x}_i$) has cost 0 for each edge incident to P_{x_i} (resp., $P_{\bar{x}_i}$). In addition, both of the have cost 0 for every edge between 2 vertices in $P_{x_i}^{dt}$'s.
- For any $j \in [m]$ and any $t \in [3]$, agent C_j^t has cost 0 for each edge incident to $P_{C_j}^{t,k}$ for all $k \in [3]$. The agent also has cost 0 for every edge between 2 vertices in $P_{C_j}^{dt}$'s.
- For an agent and an edge, if the cost of the edge for the agent is not covered by the above two sentences, then the cost is 1.

One can verify that the 2P2N-3SAT instance has a satisfiable solution if and only if there exists an EF1 assignment in the job-position instance. $\square$